Raktárkezelő rendszer

Szoftverfejlesztés és -tesztelés vizsgaremek dokumentáció

Készítette: Varga Nátán, Gersei Gábor
Osztály: Szoftverfejlesztő és -tesztelő 121/25T
Tanév: 2025/26

Tartalomjegyzék

1. Bevezetés és cél
2. Funkcionális áttekintés
3. Technológiai környezet
4. Adatbázis és backend
5. Felhasználói felület képernyőképekkel
6. Kiemelt kódrészletek magyarázattal
7. Tesztelési fejezet tesztképekkel
8. Összegzés

1. Bevezetés és cél

A projekt célja egy webalapú raktárkezelő rendszer elkészítése. A rendszer valós problémára ad megoldást: segíti a termékek, készletek, raktárak, tárolóhelyek és felhasználók áttekinthető kezelését. A manuális nyilvántartások helyett a felhasználó egy központi webes felületen tudja rögzíteni és ellenőrizni az adatokat.

A program több felhasználói szerepkört kezel. Ez azért fontos, mert egy raktári rendszerben nem minden felhasználónak kell ugyanazokat a műveleteket elérnie. Az adminisztrátor felhasználókat és jogosultságokat kezelhet, a raktáros a készletekkel dolgozhat, az olvasó szerepkör pedig elsősorban megtekintésre használható.

A megoldás asztali és mobil eszközön is használható. Mindkét nézetben a kártyás megjelenítés ad kényelmes használatot.

2. Funkcionális áttekintés

Modul	Feladat	Rövid magyarázat
Termékkezelés	Létrehozás, módosítás, törlés	A termékek alapadatai, kategóriái és képei kezelhetők.
Készletkezelés	Raktár és tárolóhely szerinti nyilvántartása	A mennyiségek raktáranként követhetők.
Felhasználók	Szerepkörök és profiladatok kezelése	A jogosultságok szabályozzák a hozzáférést.
Naplózás	Műveletek rögzítése	A rendszer eseményei később visszanézhetők.

A funkciók együtt egy olyan rendszert alkotnak, amely alkalmas alapvető raktári adminisztrációra. A rendszer nem csak adatokat tárol, hanem felhasználói szerepkörök alapján szabályozza, hogy ki milyen műveletet végezhet.

3. Technológiai környezet

A szerveroldali rész PHP nyelven készült. Az adatbázis-kezelés PDO használatával történik, ami lehetőséget ad prepared statementek alkalmazására és biztonságosabb SQL műveletekre. Az adatokat MySQL adatbázis tárolja.

A kliensoldali felület HTML, CSS és JavaScript alapú. A JavaScript a dinamikus működést biztosítja, például a keresést, kategóriaszűrést, képelőnézetet, dark mode váltást és AJAX alapú tárolóhely betöltést.

A rendszer klasszikus PHP fájlalapú felépítést használ: külön vannak a megjelenítő oldalak, az include állományok és az action fájlok. Ez a szerkezet átláthatóvá teszi, melyik fájl melyik feladatért felelős.

4. Adatbázis és backend

Az adatbázis a termékekhez, készletekhez, raktárakhoz és felhasználókhoz kapcsolódó adatokat tárolja. A relációs felépítés lehetővé teszi, hogy egy termék több raktárban vagy tárolóhelyen is megjelenjen, miközben az adatok központilag kezelhetők.

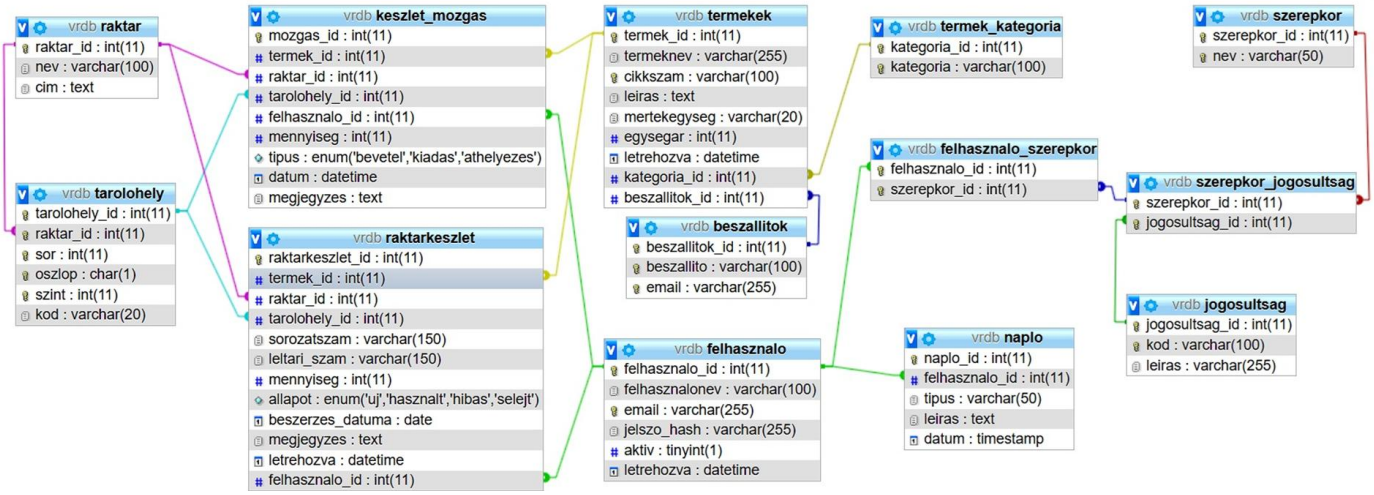
A backendben az adatbázis kapcsolat központi fájlban jön létre. Ez előnyös, mert az adatbázis beállításait egy helyen kell módosítani, és a többi oldal ezt a kapcsolatot használhatja.

Adatbázis kapcsolat kódrészlet

```
$pdo = new PDO(
    "mysql:host={$host};dbname={$dbname};charset=utf8mb4",
    $username,
    $password,
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

Magyarázat: a PDO kapcsolat UTF-8 karakterkészlettel jön létre, ezért a magyar ékezetes karakterek kezelése is biztosított. Az ERRMODE_EXCEPTION beállítás miatt hiba esetén kivétel keletkezik, ami könnyebb hibakeresést tesz lehetővé. Az emulált prepared statementek kikapcsolása biztonságosabb adatbázis-kezelést eredményez.

Adatmodell, adatkapcsolatok:



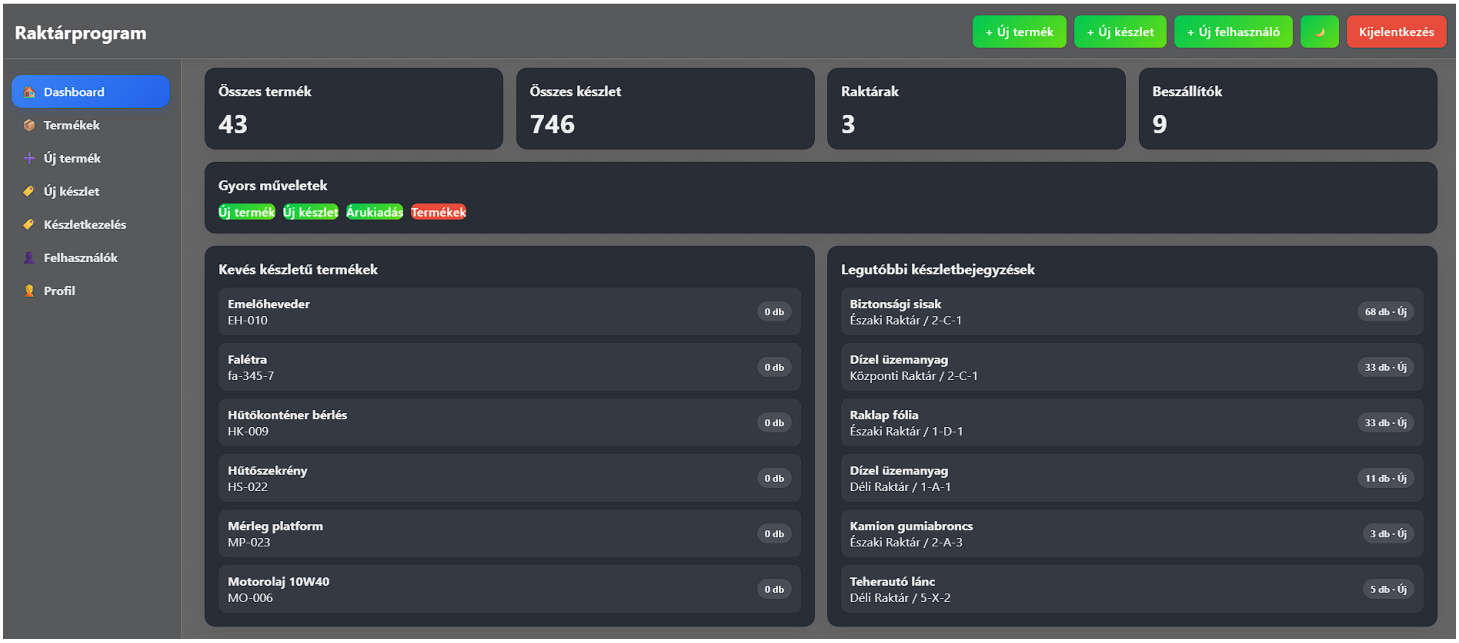
A program használatához előzetesen létrehozott tesztfelhasználók belépéséhez szükséges adatok

Szerepkör: Fellhasználónév (email cím) /jelszó:

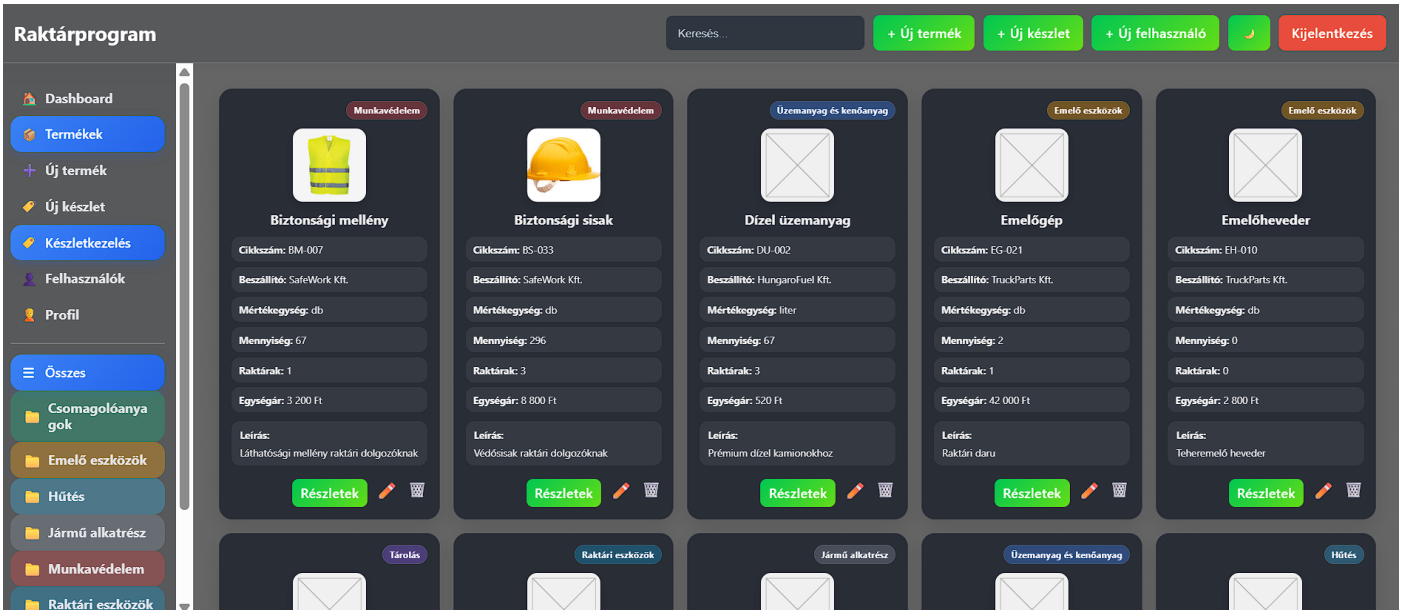
- Admin: admin@gmail.com / 123456
- Olvasó: gabor@gmail.com / 123456
- Raktáros: ferenc@gmail.com / 123456

5. Felhasználói felület képernyőképekkel

Ebben a fejezetben a program főbb képernyőinek bemutatása szerepel.

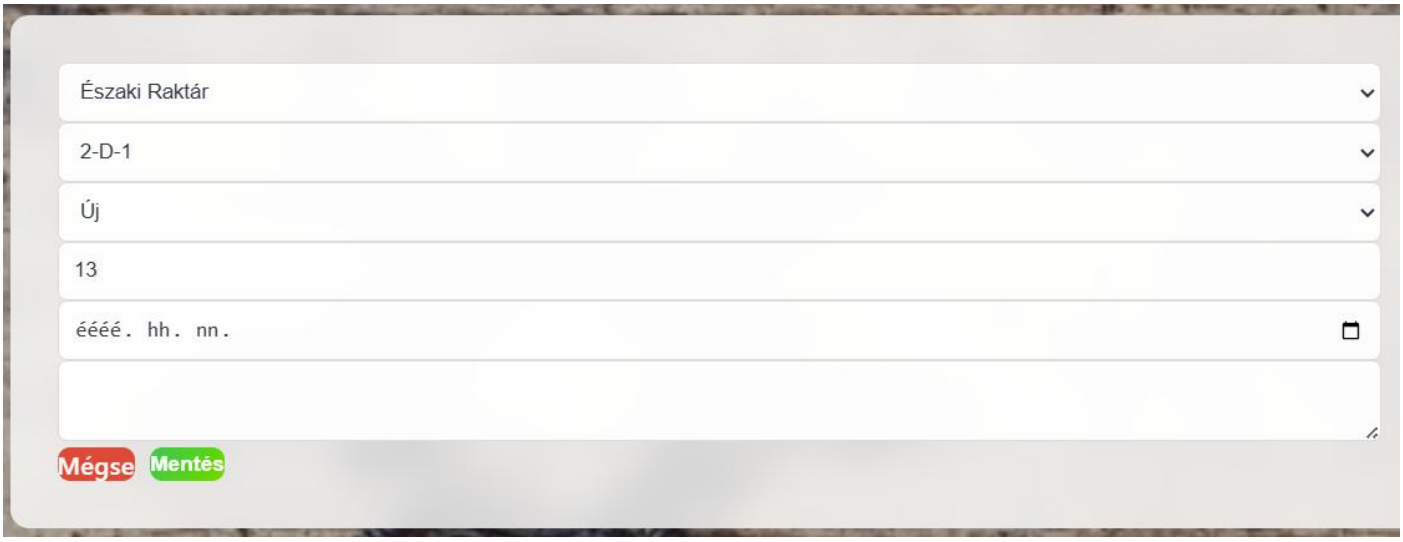


Képernyőfotó magyarázata: a főoldalon látható navigációs sáv, oldalsó menü és az összefoglaló kártyák azt mutatják, hogy a rendszer több modulból áll. A dashboard célja, hogy a felhasználó gyorsan áttekintse az aktuális állapotot.



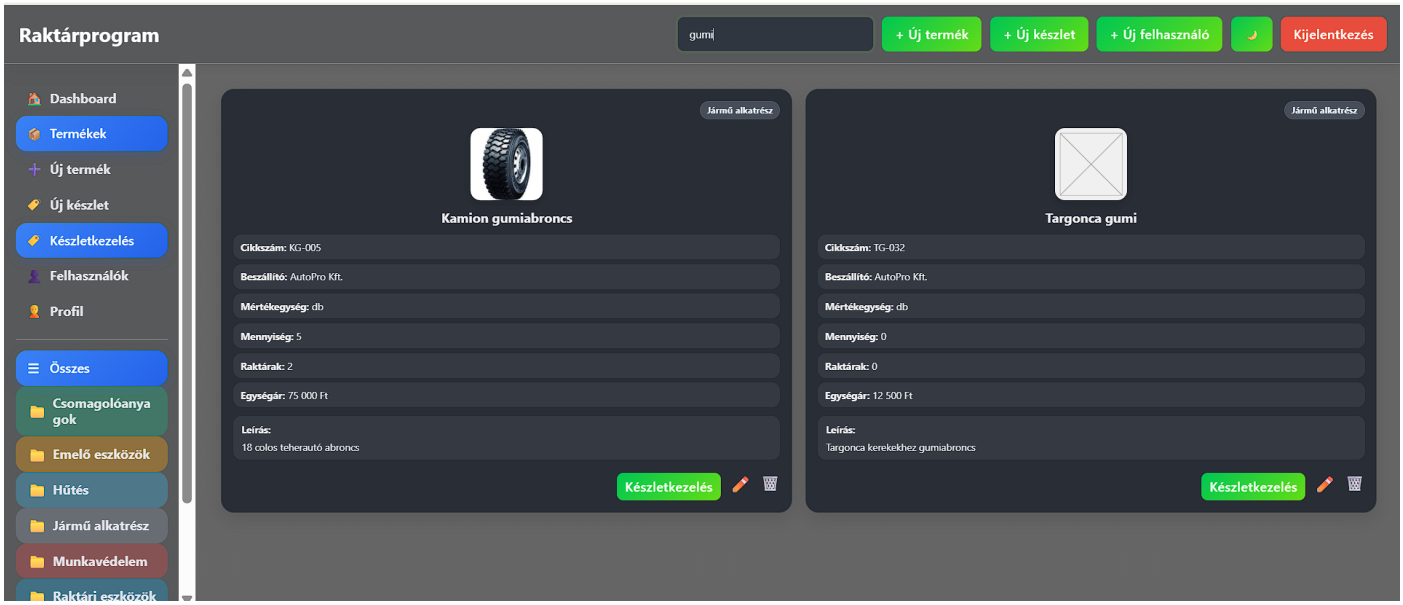
Képernyőfotó magyarázata: a terméklista bemutatja a termékek nevét, kategóriáját, képét és kezelőgombjait.

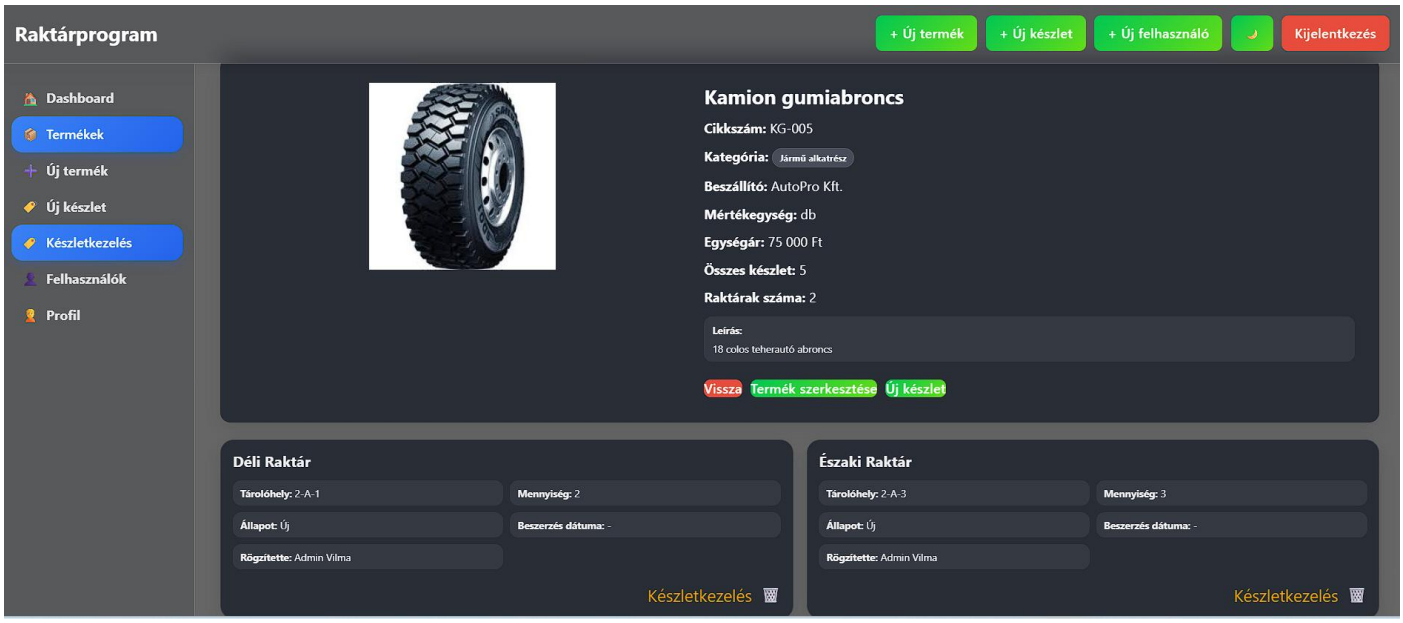
5.1 Készletkezelés képernyőkép

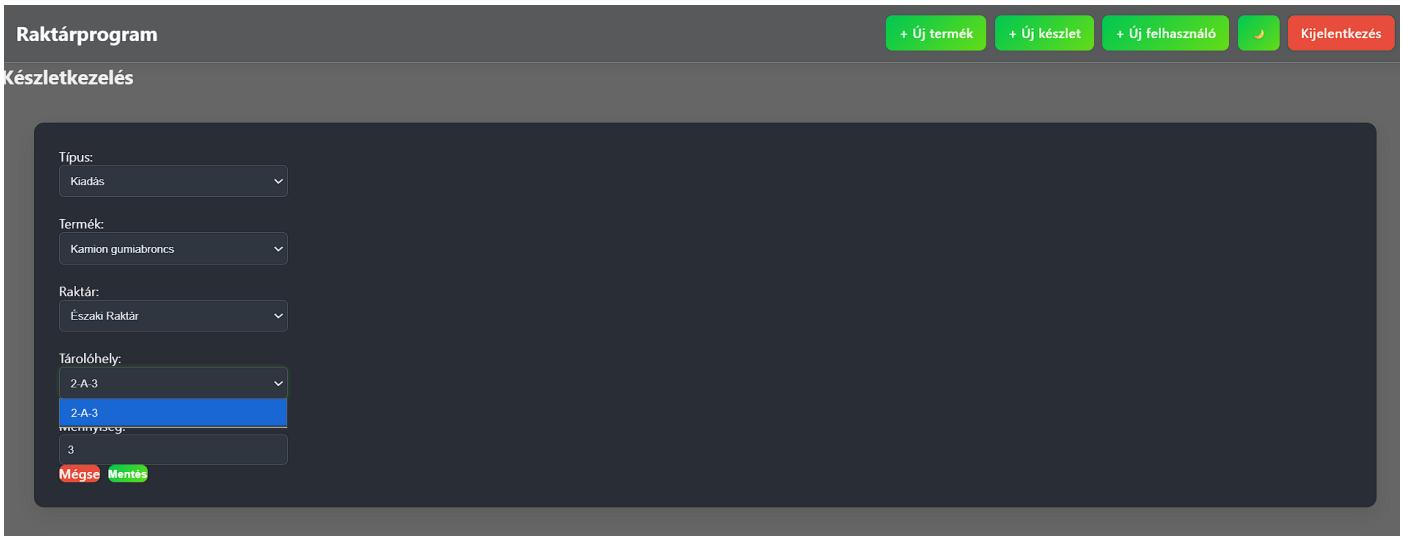


Képernyőfotó magyarázata: A módosítási és törlési lehetőségek azt mutatják, hogy a rendszer CRUD műveleteket valósít meg.

Az alább látható néhány képernyőfotón a készlet kezelésének egy alternatívája látható.
A készletkezelést, árufelvételt, árukiadást megvalósíthatjuk a főoldalon látható oldalsó menüből kiindulva a "Készletkezelés" feliratú hivatkozás segítségével is. A megjelenő terméklista valamelyik elemét választva és követve a készletkezelésre utaló hivatkozásokat eljuthatunk egy oldalra, ahol a raktárkészlet bevételezése és kiadása megtörténhet.
A kiadás esetén a lenyílólistában szűrve láthatjuk azokat a tárolóhelyeket, amelyekben a kérdéses raktárban ténylegesen tároljuk az adott árut.


Raktárprogram
gumi
+ Új termék
+ Új készlet
+ Új felhasználó
Kijelentkezés
Dashboard
Termékek
Új termék
Új készlet
Készletkezelés
Felhasználók
Profil
Összes
Csomagolóanyagok
Emelő eszközök
Hűtés
Jármű alkatrész
Munkavédelem
Raktári eszközök
Jármű alkatrész
Kamion gumiabroncs
Cikkszám: KG-005
Beszállító: AutoPro Kft.
Mértékegység: db
Mennyiség: 5
Raktárak: 2
Egységár: 75 000 Ft
Leírás:
18 colos teherautó abroncs
Készletkezelés
Jármű alkatrész
Targonca gumi
Cikkszám: TG-032
Beszállító: AutoPro Kft.
Mértékegység: db
Mennyiség: 0
Raktárak: 0
Egységár: 12 500 Ft
Leírás:
Targonca kerekekhez gumiabroncs
Készletkezelés


Raktárprogram
+ Új termék
+ Új készlet
+ Új felhasználó
Kijelentkezés
Dashboard
Termékek
Új termék
Új készlet
Készletkezelés
Felhasználók
Profil
Kamion gumiabroncs
Cikkszám: KG-005
Kategória: Jármű alkatrész
Beszállító: AutoPro Kft.
Mértékegység: db
Egységár: 75 000 Ft
Összes készlet: 5
Raktárak száma: 2
Leírás:
18 colos teherautó abroncs
Vissza
Termék szerkesztése
Új készlet
Déli Raktár
Tárolóhely: 2-A-1
Mennyiség: 2
Állapot: Új
Beszerzés dátuma: -
Rögzítette: Admin Vilma
Készletkezelés
Északi Raktár
Tárolóhely: 2-A-3
Mennyiség: 3
Állapot: Új
Beszerzés dátuma: -
Rögzítette: Admin Vilma
Készletkezelés


Raktárprogram
+ Új termék
+ Új készlet
+ Új felhasználó
Kijelentkezés
Készletkezelés
Típus:
Kiadás
Termék:
Kamion gumiabroncs
Raktár:
Északi Raktár
Tárolóhely:
2-A-3
2-A-3
3
Mégse
Mentés

6. Kiemelt kódrészletek magyarázattal

6.1 Kliens oldali szűrés

A termékek szűrése JavaScript segítségével történik. A szűrés két feltételt vizsgál: a kiválasztott kategóriát és a keresőmezőbe írt szöveget. A megoldás előnye, hogy a felhasználónak nem kell megvárnia az oldal újratöltését.

```
function applyFilters() {
    document.querySelectorAll("#productTable tbody tr").forEach(row => {
        const rowCategory = (row.dataset.category || "").toLowerCase();
        const rowText = row.innerText.toLowerCase();

        const matchesCategory =
            activeCategory === "all" || rowCategory === activeCategory.toLowerCase();

        const matchesSearch = rowText.includes(searchText);

        row.style.display = (matchesCategory && matchesSearch) ? "" : "none";
    });
}
```

Magyarázat: a függvény végigmegy a táblázat sorain. A dataset.category alapján ellenőrzi a kategóriát, az innerText alapján pedig a keresett szöveget. Ha mindkét feltétel teljesül, a sor látható marad, különben elrejtésre kerül.

6.2 AJAX alapú tárolóhely betöltés

A raktár kiválasztása után a tárolóhelyek dinamikusan töltődnek be. Ez AJAX működés, mert az oldal nem tölt újra, csak a szükséges adatok érkeznek meg a szerverről JSON formában.

```
fetch('ajax/get_tarolohelyek.php?raktar_id=' + encodeURIComponent(raktarId))
    .then(response => response.json())
    .then(data => {
        taroloSelect.innerHTML = '<option value="">Tárolóhely kiválasztása</option>';

        data.forEach(item => {
            const option = document.createElement('option');
            option.value = item.tarolohely_id;
            option.textContent = item.kod;
            taroloSelect.appendChild(option);
        });
    })
    .catch(() => {
        taroloSelect.innerHTML = '<option value="">Hiba a betöltés során</option>';
    });
```

Magyarázat: a fetch kérés elküldi a kiválasztott raktár azonosítóját a PHP végpontnak. A szerver JSON választ küld vissza, amelyből a JavaScript option elemeket hoz létre. Hiba esetén a felület tájékoztató üzenetet jelenít meg.

6.3 Képfeltöltés előnézet

A kiválasztott képekről a böngésző előnézetet készít, így a felhasználó már mentés előtt ellenőrizheti, hogy megfelelő fájlokat választott-e ki.

```
const reader = new FileReader();

reader.onload = function (e) {
    const wrapper = document.createElement("div");
    wrapper.classList.add("preview-box");

    const img = document.createElement("img");
    img.src = e.target.result;
    img.classList.add("preview-item");
    img.alt = file.name;

    wrapper.appendChild(img);
    previewGallery.appendChild(wrapper);
};

reader.readAsDataURL(file);
```

Magyarázat: a FileReader API a kiválasztott fájlt a böngészőben olvassa be, majd az eredményből kép előnézet készül. Ez még nem jelenti azt, hogy a fájl már a szerveren van; az előnézet kliens oldalon történik.

6.4 Jogosultság és CSRF védelem

A rendszer session alapú bejelentkezést használ. A CSRF token célja, hogy az űrlapok beküldésénél ellenőrizhető legyen, valóban a saját alkalmazásból indult-e a kérés.

```
function ensure_csrf_token(): string
{
    if (empty($_SESSION['csrf_token']) || !is_string($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }

    return $_SESSION['csrf_token'];
}

function validate_csrf_token($token): bool
{
    if (!is_string($token) || $token === '') {
        return false;
    }

    $sessionToken = $_SESSION['csrf_token'] ?? null;
    return is_string($sessionToken) && hash_equals($sessionToken, $token);
}
```

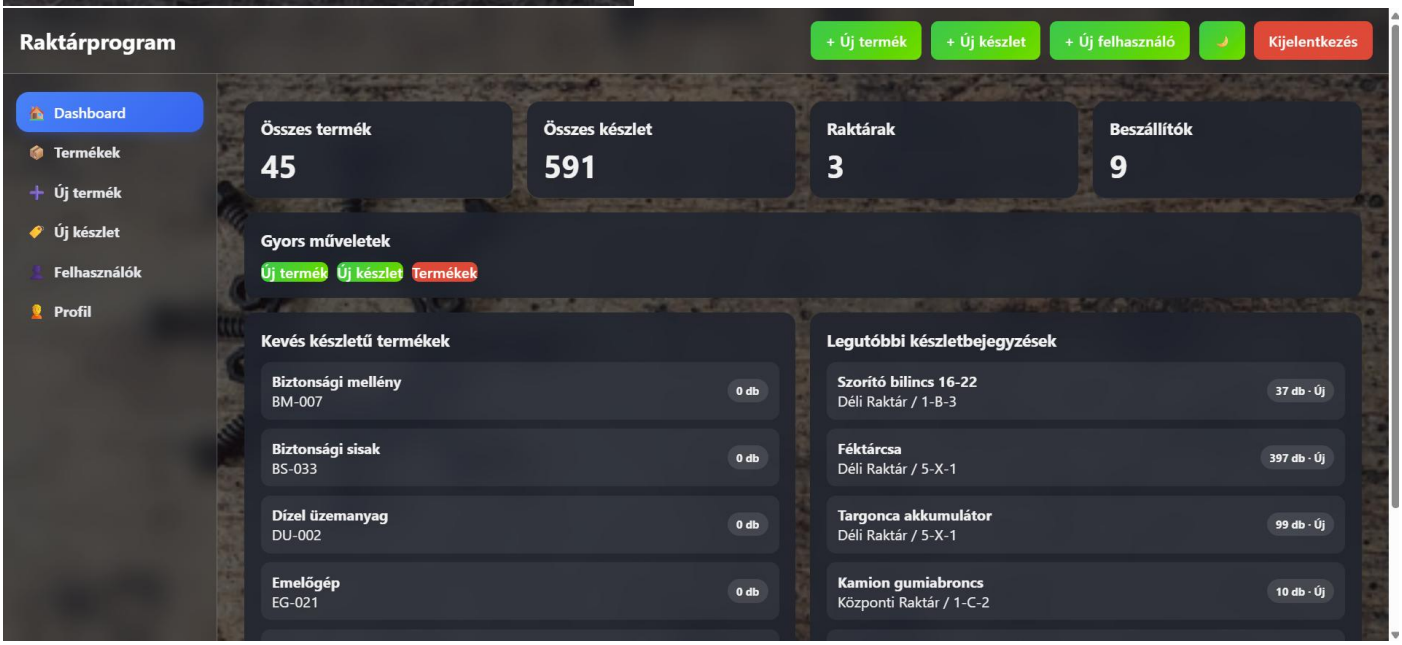
Magyarázat: a token véletlenszerű bájtokból készül, majd a sessionben tárolódik.
Beküldéskor a rendszer összehasonlítja az űrlapban érkező tokent a sessionben tárolttal. A hash_equals biztonságos összehasonlítást végez.

7. Tesztelési fejezet tesztképekkel

A tesztelés célja annak ellenőrzése volt, hogy a program fő funkciói a felhasználói elvárásoknak megfelelően működnek. A tesztelés manuálisan történt, több különböző használati esettel. A dokumentációban minden teszthez érdemes képernyőképet csatolni.

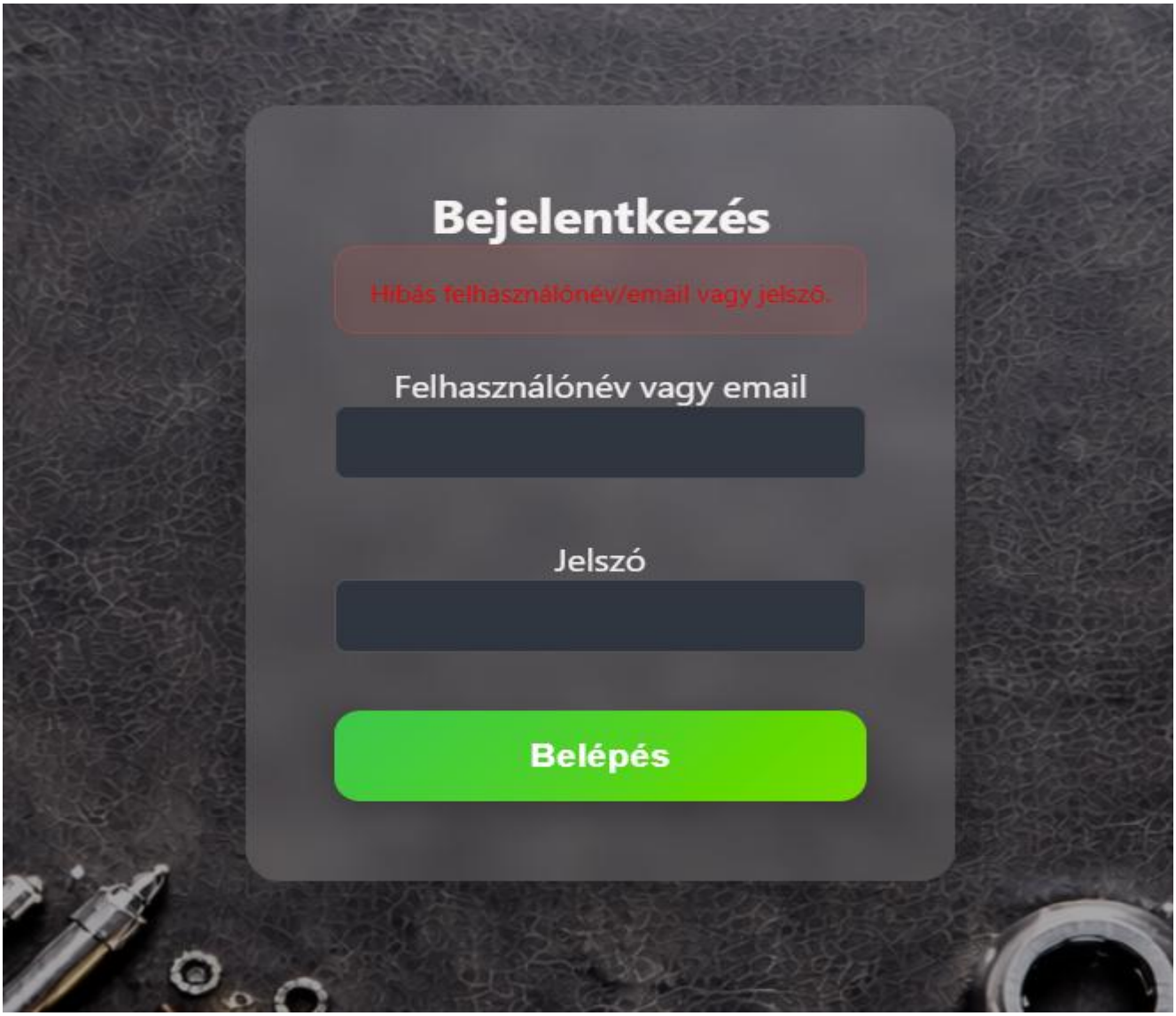
Teszt azonosító	Funkció	Elvárt eredmény	Eredmény
T-01	Bejelentkezés helyes adatokkal	A felhasználó belép a rendszerbe	Sikeres
T-02	Bejelentkezés hibás jelszóval	Hibaüzenet jelenik meg	Sikeres
T-03	Új termék létrehozása	A termék megjelenik a listában	Sikeres
T-04	Kategóriaszűrés	Csak a megfelelő termékek látszanak	Sikeres
T-05	Raktár kiválasztása	Betöltődnek a tárolóhelyek	Sikeres

7.1 Tesztkép: sikeres bejelentkezés



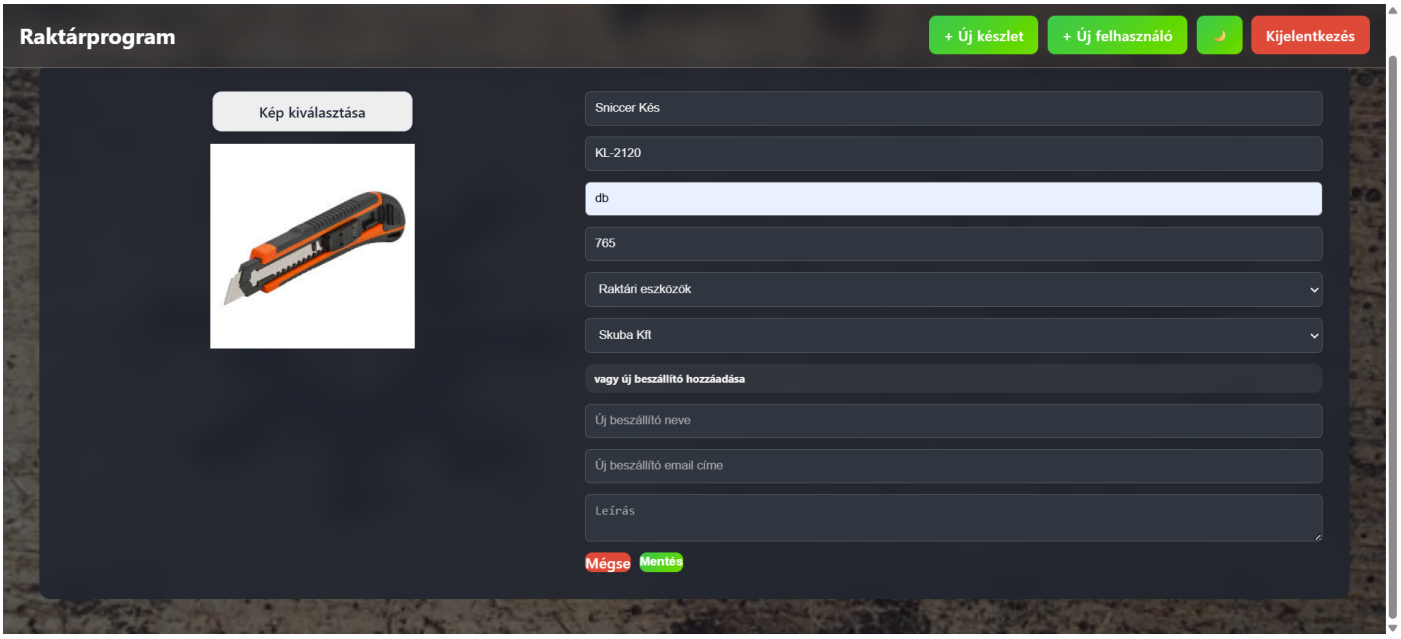
Teszt magyarázata: a teszt során helyes felhasználói adatokat kell megadni. Sikeres működés esetén a rendszer létrehozza a sessiont, és a felhasználó belép a védett felületre.

7.2 Tesztkép: hibás bejelentkezés



Teszt magyarázata: ez a teszt a hibakezelést ellenőrzi. A rendszernek nem szabad beengednie a felhasználót hibás adatokkal. A cél, hogy a felhasználó érthető visszajelzést kapjon.

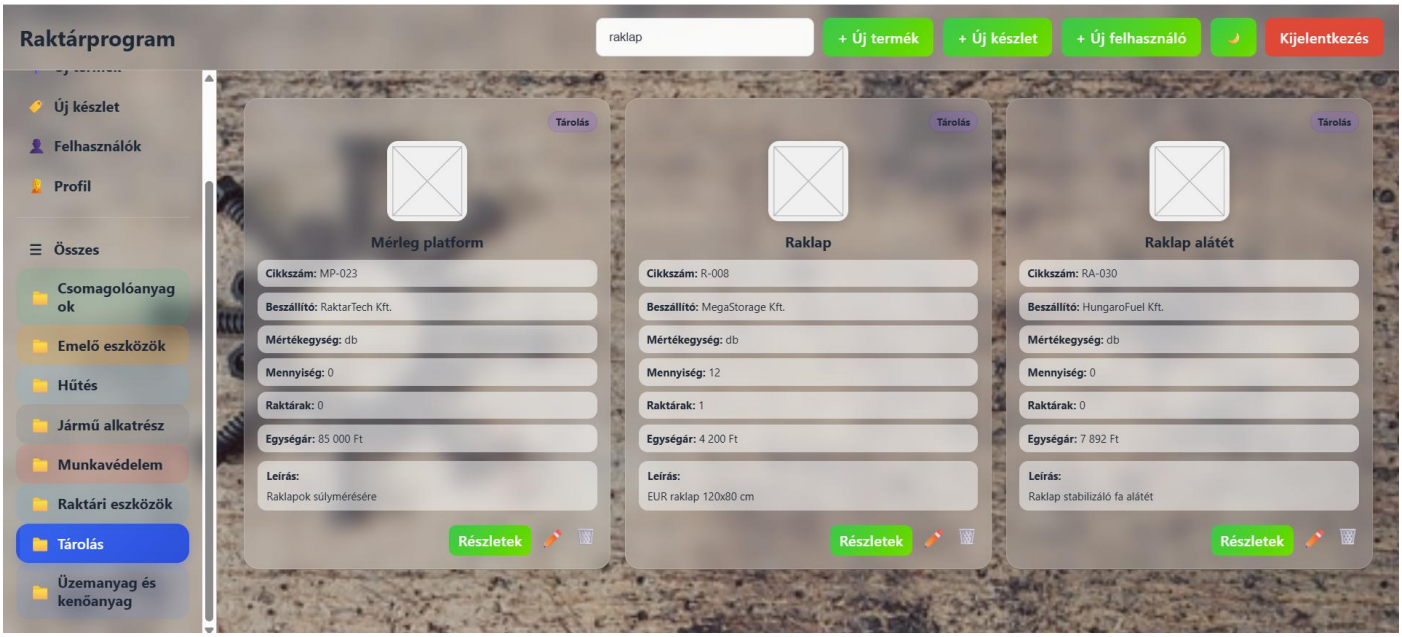
7.3 Tesztkép: termék létrehozása



Teszt magyarázata: a teszt során ki kell tölteni a kötelező mezőket, majd elmenteni a terméket. Sikeres teszt esetén az új rekord bekerül az adatbázisba, és a felületen is látható lesz.

7.4 Tesztkép: Szűrés és Keresés

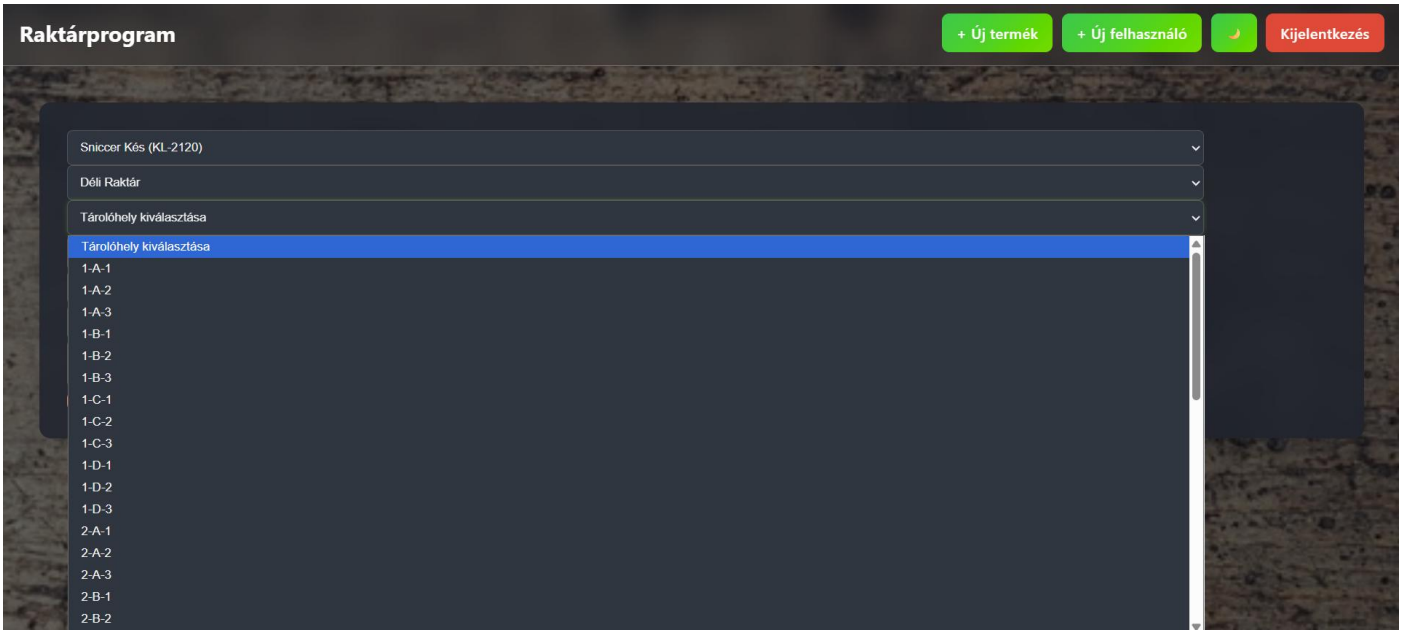
Teszt magyarázata: A képen a kategória szerinti szűrés működése látható. A kiválasztott kategóriának megfelelő termékek jelennek meg, míg a többi elem rejtve marad.



Teszt magyarázata: A teszt során kiválasztottam a „Tárolás” kategóriát, majd a keresőmezőbe beírtam egy kulcsszót. Ezt követően ellenőriztem, hogy a rendszer csak azokat a termékeket jeleníti-e meg, amelyek megfelelnek mind a kategóriának, mind a keresési feltételnek.

A képen látható, hogy a szűrés megfelelően működik, mivel kizárólag releváns elemek jelennek meg. Ez igazolja, hogy a két szűrési mechanizmus együttesen is helyesen működik.

7.5 Tesztkép: AJAX tárolóhely betöltés



Teszt magyarázata: a teszt a kliens és szerver közötti aszinkron kommunikációt ellenőrzi. A sikeres működés azt jelenti, hogy a fetch kérés választ kap a PHP végponttól, és a JavaScript frissíti a select mezőt.

8. Összegzés

A projekt eredményeként egy működő webes raktárkezelő rendszer készült. A program tartalmaz termékkezelést, készletkezelést, felhasználókezelést, jogosultságkezelést, képfeltöltést, naplózást és dinamikus felületi elemeket.

A dokumentációban bemutatott kódrészletek azt szemléltetik, hogy a rendszer nem csak statikus oldalakból áll, hanem kliens oldali logikát, szerveroldali adatkezelést és aszinkron kommunikációt is használ. A tesztelési fejezet a fő funkciók ellenőrzéséhez ad rendszerezett alapot.